

PayNexus: Uma Carteira Digital Distribuída em Microserviços

Relatório Técnico - Engenharia e Gestão de Serviços (EGS)

Grupo 3 - DETI, Universidade de Aveiro

Pedro Melo, n.º XXXXXX

Simão Almeida, n.º XXXXXX

Tiago Garcia, n.º XXXXXX

Rúben Costa, n.º 114190

Engenharia de Computadores e Telemática

Junho de 2026

Resumo

Este relatório descreve o projeto **PayNexus**, uma plataforma de carteira digital implementada como um conjunto de microsserviços independentes. O problema central abordado é o carregamento direto de dinheiro para a *blockchain*: um utilizador paga com cartão de crédito (Stripe), o pagamento é confirmado por *webhook* com verificação HMAC e protegido por código OTP via WhatsApp (Twilio), e o valor é imediatamente creditado numa carteira Ethereum na testnet Sepolia via Web3j, terminando com uma notificação *Web Push* em tempo real. O sistema é composto por seis serviços independentes (IAM, Pagamentos, Transações, Notificações e dois *frontends*) escritos em quatro linguagens (Python, Java, Go, TypeScript), com base de dados própria por serviço (PostgreSQL 15 e Redis 7), *gateway* Traefik v3.1, gestão centralizada de segredos com HashiCorp Vault 1.17, autenticação OAuth2/OIDC via Keycloak 26.1 em dois *realms*, e orquestração em Docker Compose (desenvolvimento) e Kubernetes (produção no cluster DETI).

Conteúdo

1	Introdução	3
2	Visão Geral do Sistema	3
3	Arquitetura de Microserviços	5
3.1	Princípios Aplicados	5
3.2	Catálogo de Serviços	5
4	Stack Tecnológica	6

5	Implementação dos Serviços	7
5.1	IAM Service (Serviço de Identidade)	7
5.2	Payment Service (Serviço de Pagamentos)	7
5.3	Transactions Service (Serviço de Transações e Blockchain)	8
5.4	Notifications Service (Serviço de Notificações)	9
5.5	Frontends (React)	9
6	Comunicação Inter-Serviços	10
7	Segurança	11
7.1	Autenticação OAuth2/OIDC com Keycloak	11
7.2	Gestão de Segredos com HashiCorp Vault	12
7.3	Outros Mecanismos de Segurança	13
8	Containerização e Orquestração	13
8.1	Docker Compose (Desenvolvimento)	13
8.2	Kubernetes (Produção)	14
9	Observabilidade	15
10	Desafios e Conclusões	16
10.1	Desafios Encontrados	16
10.2	Conclusões	16

1 Introdução

O projeto **PayNexus** nasce com o propósito de democratizar o acesso ao ecossistema *blockchain*, eliminando as barreiras técnicas tradicionais de entrada no mundo Web3. A nossa proposta de valor centra-se na criação de uma ponte fluida e segura entre o sistema financeiro tradicional (Web2) e as redes descentralizadas (Web3), permitindo que qualquer utilizador carregue e movimente saldo *on-chain* recorrendo simplesmente a um cartão de crédito.

Para viabilizar este modelo de negócio complexo — que exige a orquestração de domínios com ciclos de vida e modos de falha completamente distintos — afastámo-nos de uma abordagem monolítica tradicional. Em vez disso, o **PayNexus** foi desenvolvido com uma arquitetura baseada de **microsserviços** independentes. Cada serviço está isolado e foca-se num domínio específico do sistema, o que facilita a manutenção e reduz dependências entre componentes. Esta abordagem vai ao encontro dos objetivos curriculares da unidade de Engenharia e Gestão de Serviços (EGS): a arquitetura em si é o *deliverable* principal, demonstrando princípios como *bounded contexts*, implantação independente, desenvolvimento polígota, e orquestração com contentores.

Objetivos do projeto:

- Implementar um fluxo completo de pagamento com cartão e crédito em *blockchain*;
- Demonstrar princípios de microsserviços com tecnologias heterogéneas;
- Garantir segurança por camadas: autenticação, segredos em *runtime*, isolamento de rede;
- Disponibilizar observabilidade operacional via dashboards Grafana;
- Orquestrar o sistema em Docker Compose (desenvolvimento) e Kubernetes (produção).

Este relatório está organizado da seguinte forma: a Secção 2 apresenta uma visão geral do sistema; a Secção 3 detalha a arquitetura de microsserviços; a Secção 4 descreve a *stack* tecnológica; a Secção 5 documenta a implementação de cada serviço; a Secção 6 analisa a comunicação inter-serviços; a Secção 7 cobre os mecanismos de segurança; a Secção 8 descreve a containerização e orquestração; a Secção 9 aborda a observabilidade; e a Secção 10 apresenta os desafios e conclusões.

2 Visão Geral do Sistema

A plataforma PayNexus é uma **carteira digital distribuída**: os utilizadores autenticam-se, carregam a carteira com um pagamento real por cartão, movem fundos como transações *blockchain*, e recebem notificações em tempo real, tudo entregue por um conjunto de microsserviços independentes.

Tabela 1: Resumo do projeto PayNexus

Aspeto	Descrição
Domínio	Carteira digital / pagamentos / transações <i>blockchain</i>
Arquitetura	Microserviços com <i>gateway</i> único
Microserviços	6 (4 <i>backends</i> + 2 <i>frontends</i>)
Linguagens	4Python, Java, Go, TypeScript
Autenticação	Keycloak (OAuth2/OIDC), dois <i>realms</i>
Pagamentos	Stripe (PaymentIntent + <i>webhooks</i>) + OTP Twilio
<i>Blockchain</i>	Ethereum Sepolia testnet via Web3j
Tempo real	Notificações Web Push (multi- <i>tenant</i>)
Dados	PostgreSQL 15 por serviço + Redis 7
<i>Gateway</i>	Traefik v3.1 (apenas porta 80 exposta)
Segredos	HashiCorp Vault (AppRole + <i>agent sidecars</i>)
Observabilidade	Dashboards Grafana 10
Desenvolvimento	Docker Compose (≈ 17 contentores)
Produção	Kubernetes (cluster DETI/UA)

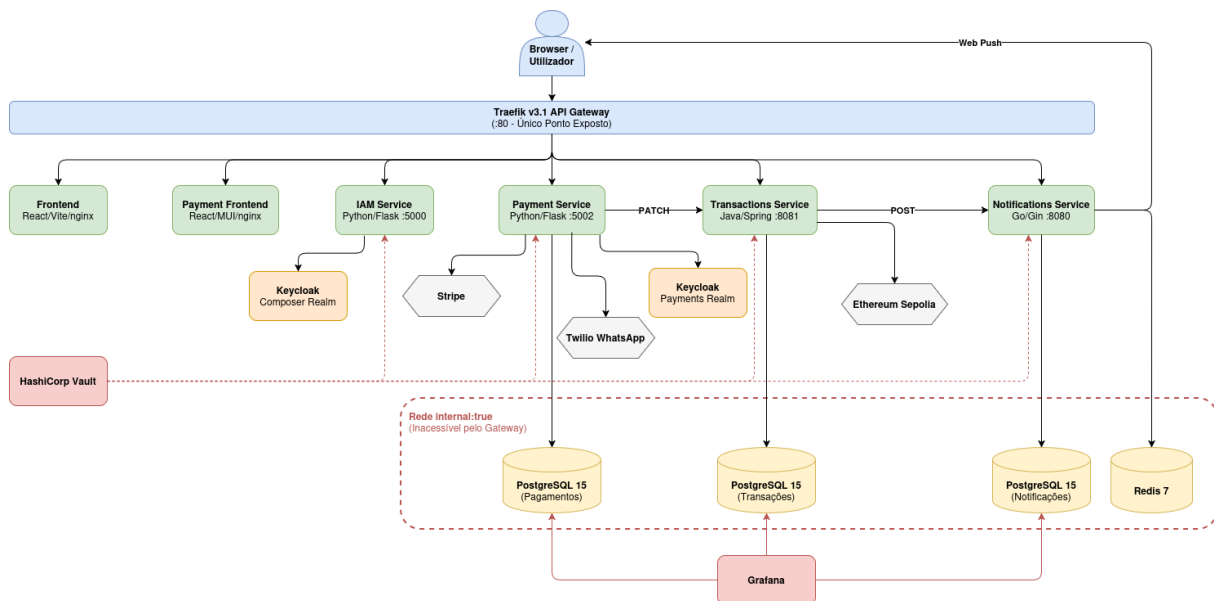


Figura 1: Arquitetura Geral do Sistema

O roteamento do *gateway* Traefik é baseado em *headers* de *host* HTTP, conforme indicado na Tabela 2.

Tabela 2: Roteamento via Traefik (desenvolvimento local)

Host	Serviço	Porta interna
app.pt	Frontend principal	80
payment.pt	Payment frontend	5174
iam.pt	IAM service	5000
transactions.pt	Transactions service	8081
payment-api.pt	Payment backend	5002
notifications.pt	Notifications service	8080
keycloak.pt	Keycloak (compositor)	8080
payment-keycloak.pt	Keycloak (pagamentos)	8080
grafana.pt	Grafana	3000

3 Arquitetura de Microserviços

3.1 Princípios Aplicados

A arquitetura do PayNexus foi desenhada com os princípios fundamentais de microserviços em mente. A Tabela 3 descreve como cada princípio foi concretizado no projeto.

Tabela 3: Princípios de microserviços e aplicação no projeto

Princípio	Aplicação no PayNexus
<i>Bounded contexts</i>	Cada serviço possui um domínio exclusivo: identidade, pagamentos, transações, notificações
Implantação independente	Cada serviço tem o seu próprio <code>Dockerfile</code> , imagem e ciclo de vida
Desenvolvimento polígloa	Python (APIs web rápidas), Java/Spring (blockchain + tipagem forte), Go (notificações de alto débito)
Base de dados por serviço	Sem esquema partilhado; instância PostgreSQL isolada por serviço em rede própria
Isolamento de falhas	A falha de um serviço não compromete a disponibilidade global do sistema, graças ao isolamento entre serviços e redes independentes
Ponto de entrada único	O Traefik esconde a topologia interna atrás de uma única porta

3.2 Catálogo de Serviços

A Tabela 4 apresenta o catálogo completo dos seis serviços que compõem a plataforma.

Tabela 4: Catálogo de microsserviços do PayNexus

Serviço	Responsabilidade	Framework	Porta	Bibliotecas-chave
iam_service	Registo, login e validação de <i>tokens</i> via Keycloak	Python/Flask 3.1	5000	python-keycloak, Flask
payment_service	Pagamentos com cartão, OTP, recibos, cartões guardados	Python/Flask 3.1	5002	Stripe 14.4, Twilio, SQLAlchemy, ReportLab
transactions_service	Carteiras, transações <i>blockchain</i> , composição BFF	Java 21 / Spring Boot 3.4.3	8081	Web3j 4.10, Spring Data JPA, SpringDoc
notifications_service	Clientes, subscrições, Web Push em tempo real	Go/Gin	8080	gin, gorm, webpush-go, jwt
frontend	UI principal da carteira e histórico	React 18 + TypeScript + Vite	80	React Router, Axios, React Query
payment frontend	UI de pagamento dedicada	React 18 + TypeScript + MUI	5174	@stripe/react-stripe-js, MUI

4 Stack Tecnológica

A Tabela 5 resume todas as tecnologias utilizadas na plataforma, organizadas por camada funcional.

Tabela 5: Stack tecnológica completa do PayNexus

Camada	Tecnologia	Versão / Notas
<i>Frontend</i>	React + TypeScript + Vite	React 18.2, Vite 5.5, servido via nginx
	Material-UI (MUI)	7.3.9 biblioteca de componentes UI
	Axios, React Query, React Router	HTTP, <i>data fetching</i> , roteamento
<i>Backend</i>	Python 3.11 + Flask 3.1	IAM e Payment services
	Java 21 + Spring Boot 3.4.3	Transactions service
	Go + Gin 1.10	Notifications service
Dados	PostgreSQL 15	Uma instância por serviço
	Redis 7	Notificações - cache e pub/sub
	ORMs	SQLAlchemy, Spring Data JPA, GORM
Autenticação	Keycloak 26.1	Dois <i>realms</i> , OAuth2/OIDC, JWT
<i>Gateway</i>	Traefik v3.1	Roteamento por <i>host</i> , multi-instância

(continuação da Tabela 5)

Camada	Tecnologia	Versão / Notas
Contentores	Docker + Docker Compose	<i>Multi-stage builds</i>
Orquestração	Kubernetes	Cluster DETI, Traefik Ingress
Segredos	HashiCorp Vault 1.17	AppRole + <i>agent sidecars</i>
Observabilidade	Grafana 10	Datasources PostgreSQL por serviço
Pagamentos	Stripe SDK 14.4.1	PaymentIntent + <i>webhooks</i>
OTP/SMS	Twilio SDK 9.10.4	WhatsApp OTP
<i>Blockchain</i>	Web3j 4.10 + Ethereum Sepolia	Chain ID: 11155111
<i>Push</i>	Web Push API + webpush-go	VAPID por cliente <i>tenant</i>

5 Implementação dos Serviços

5.1 IAM Service (Serviço de Identidade)

O **IAM Service** é o ponto de entrada para autenticação de utilizadores. Implementado em Python/Flask 3.1, funciona como um *proxy* OIDC que delega toda a lógica de identidade ao Keycloak.

Responsabilidades:

- Gerir o fluxo de *sign-up* e *login* via Keycloak (*realm* microservices);
- Trocar o código de autorização OAuth2 por *tokens* JWT;
- Validar *tokens* em pedidos subsequentes;
- Expor documentação Swagger em `/iam/docs`.

A integração com o protocolo OIDC/OAuth2 foi desenvolvida de raiz através de chamadas HTTP diretas aos endpoints do Keycloak, utilizando a biblioteca `requests`. O estado de sessão necessário para a validação do parâmetro `state` do OAuth2 é gerido de forma *stateless* no cliente. O *backend* em Flask assina criptograficamente os dados da sessão e armazena-os num *cookie* seguro no navegador do utilizador, garantindo que o serviço permanece completamente independente de estado local e perfeitamente preparado para escalabilidade horizontal.

5.2 Payment Service (Serviço de Pagamentos)

O **Payment Service**, implementado em Python/Flask 3.1 com SQLAlchemy e PostgreSQL 15, é responsável por todo o ciclo de vida dos pagamentos com cartão.

Fluxo principal:

1. O *frontend* invoca `POST /v1/payments` para criar um `PaymentIntent` na Stripe;
2. O Stripe.js no browser confirma o pagamento diretamente com a Stripe;
3. A Stripe envia um *webhook* com assinatura HMAC para `POST /v1/payments/webhook`;

4. O serviço verifica a assinatura no cabeçalho **Stripe-Signature**;
5. É enviado um OTP de 6 dígitos via WhatsApp (Twilio), com validade de 5 minutos;
6. Após verificação do OTP, o pagamento é marcado como **concluded** e o serviço chama o Transactions Service (Tabela 6).

Tabela 6: Endpoints principais do Payment Service

Endpoint	Método	Descrição
/v1/payments	POST	Cria PaymentIntent na Stripe
/v1/payments	GET	Lista pagamentos do utilizador (Bearer JWT)
/v1/payments/{id}	PATCH	Atualiza estado do pagamento
/v1/payments/{id}/send-otp	POST	Envia OTP via Twilio WhatsApp
/v1/payments/{id}/verify	POST	Valida código OTP
/v1/payments/webhook	POST	Recebe <i>webhook</i> Stripe (HMAC)
/v1/payments/{id}/receipt	GET	Gera recibo PDF (ReportLab)
/v1/users/profile	GET/PUT	Perfil do utilizador
/v1/users/cards	GET/POST	Cartões guardados (Stripe Customer)

Modelo de dados (PostgreSQL): o serviço mantém quatro tabelas **payments** (UUID, *user_id*, *montante*, *estado*, *stripe_payment_intent_id*, *wallet_id*), **user_profiles** (*stripe_customer_id*, número de telefone), **saved_cards** (*payment_method_id*, *last4*, *marca*, *validade*) e **payment_otps** (*código* com hash SHA-256, *expiração*, *flag* de utilização).

Geração de recibos PDF: a biblioteca ReportLab gera recibos em memória (*in-memory*), com valores em EUR e tipografia Helvetica, servidos com **Content-Disposition: attachment**.

5.3 Transactions Service (Serviço de Transações e Blockchain)

O **Transactions Service** é o núcleo central da plataforma. Implementado em Java 21 com Spring Boot 3.4.3, integra-se diretamente com a *blockchain* Ethereum e atua simultaneamente como motor de carteiras e como um *API Composer/BFF* (Backend for Frontend), orquestrando o fluxo financeiro inter-serviços.

Integração blockchain (Web3j 4.10):

- **Rede:** Ethereum Sepolia testnet (chain ID: 11155111);
- **Carteiras:** Criadas por utilizador, com as chaves privadas armazenadas e encriptadas em AES recorrendo a uma chave mestra (**MASTER_KEY_FOR_WALLET** injetada em *runtime* via Vault);
- **Operações:** Consulta de saldos, estimativa de *gas*, assinatura de *Raw Transactions* e consulta assíncrona de recibos de blocos (*Receipts*);
- **Modos:** *Mock provider* para testes locais e *real provider* para integração efetiva com a rede de testes.

Papel de Composer / BFF: O serviço centraliza os pedidos do Frontend de Transações, atuando como agregador. Ele expõe endpoints para iniciar o fluxo de autenticação do operador (delegando as chamadas de *login* e *signup* ao *Payment Service* em Python) e coordena a transição de estados: validação de liquidação em Euros → transferência *on-chain* de fundos → notificação ao vivo do utilizador. As operações pesadas na *blockchain*

e as chamadas ao serviço de notificações são delegadas para execução assíncrona através de fios de execução (*threads*) independentes geridos pela anotação `@EnableAsync`.

Endpoints principais:

- **POST /v1/payments** Encaminha a solicitação de criação de uma intenção de pagamento para o *Payment Service*; se o pagamento for imediatamente retornado como concluído, despoleta de imediato a transferência de fundos *on-chain*;
- **PATCH /v1/payments/{id}** Atua como o *endpoint* de receção de notificações internas; é invocado de forma assíncrona pelo *Payment Service* (Python) após o sucesso do *webhook* da Stripe, sendo protegido por um bypass de autenticação via cabeçalho `X-Internal-Key`. Ao receber este sinal, o serviço inicia a transferência de ETH a partir da *bankWallet* para a carteira do utilizador;
- **GET /v1/payments/{id}** Consulta o detalhe e o estado atualizado de um pagamento;
- **GET /v1/pay/login** e **GET /v1/pay/signup** Disponibilizam as URLs de redirecionamento OIDC geradas pelo ecossistema de pagamentos.

5.4 Notifications Service (Serviço de Notificações)

O **Notifications Service** é um motor *Web Push* multi-tenant de uso geral, implementado em Go com o *framework* Gin. Qualquer aplicação cliente pode ser integrada em tempo de execução, recebendo uma chave API e par de chaves VAPID próprias.

Ciclo de vida de um cliente *tenant*:

1. Administrador cria cliente via **POST /v1/admin/clients**;
2. Sistema devolve chave API + chaves VAPID (pública e privada);
3. O cliente usa a chave VAPID pública para subscrever utilizadores no browser;
4. Subscrições são registadas em **POST /v1/events/subscribe**;
5. O Transactions Service publica eventos em **POST /v1/events** com *Bearer API key*;
6. O serviço entrega notificações cifradas via Web Push ao *Service Worker* do browser.

Performance: o Redis 7 é usado para cache de clientes e subscrições, reduzindo consultas à base de dados PostgreSQL nas operações de alta frequência. Os *tokens* JWT de subscrição têm validade de 15 minutos.

Segurança multi-tenant: cada cliente recebe o seu próprio par de chaves VAPID, garantindo isolamento criptográfico entre *tenants* e cumprindo a RFC 8292 [1].

5.5 Frontends (React)

A plataforma inclui dois *frontends* independentes:

Frontend principal (app.pt): SPA React 18 + TypeScript + Vite que expõe a carteira digital, histórico de transações, e gestão de conta. Comunica com o Transactions Service (BFF) para todas as operações. Servido por nginx Alpine.

Payment Frontend (payment.pt): SPA dedicada ao fluxo de pagamento, construída com React 18 + MUI 7.3 e integra o Stripe.js via `@stripe/react-stripe-js`. Isolada num Keycloak *realm* separado, garantindo que credenciais de utilizadores de pagamento nunca se misturam com o *realm* microservices.

A estratégia de *containerização* dos *frontends* seguiu duas abordagens distintas:

O **Transactions Frontend** utiliza um modelo *multi-stage*. Na primeira fase, o código é compilado e, na fase final, os ficheiros estáticos são transferidos para um servidor **Nginx**, que os serve de forma leve no porto 80.

Por outro lado, o **Payment Frontend** corre diretamente em modo de desenvolvimento através do servidor nativo do **Vite** (NodeJS) no porto 5174, sem recurso ao Nginx. Ambos os serviços são geridos e encaminhados para o exterior pelo **Traefik** com base no domínio de acesso.

6 Comunicação Inter-Serviços

Todos os serviços comunicam via **REST/HTTP** sobre as redes internas Docker/Kubernetes. São usados dois padrões de autenticação: *tokens* **JWT Bearer** para chamadas orientadas ao utilizador, e o cabeçalho **X-Internal-Key** para chamadas de serviço-a-serviço que não devem expor credenciais de utilizador.

Tabela 7: Chamadas inter-serviços no PayNexus

Chamada	De → Para	Endpoint	Autenticação
Concluir pagamento	Payment → Transactions	PATCH /v1/payments/{id}	X-Internal-Key
Emitir evento	Transactions → Notifications	POST /v1/events	Bearer <api-key>
Login/tokens	Frontend → IAM → Keycloak	Fluxo OIDC code	OAuth2
Entrega Web Push	Notifications → Browser	Web Push cifrado	Chaves VAPID
Webhook Stripe	Stripe → Payment	POST /v1/payments/webhook	HMAC Stripe-Signature

Uma consequência arquitetural importante é a **consistência eventual**: um pagamento só é marcado como “concluído” após o *webhook* da Stripe ser recebido e verificado não quando o utilizador clica em “pagar”. Isto significa que o sistema aceita um período de latência entre a ação do utilizador e a atualização de estado, em troca de garantia criptográfica de que o pagamento foi efetivamente processado.

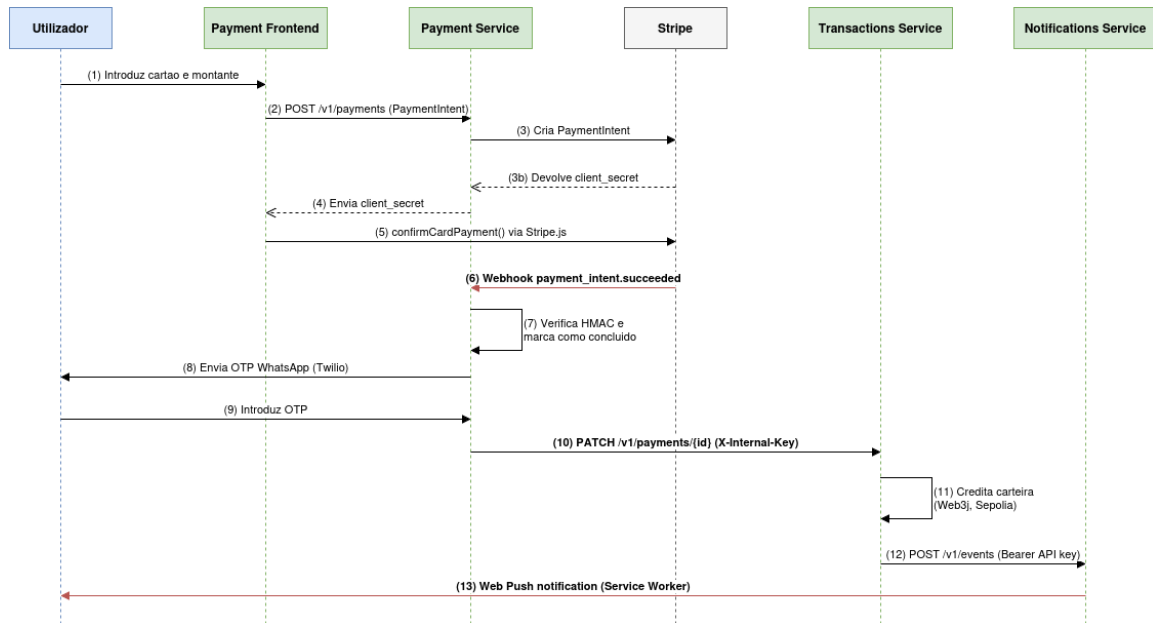


Figura 2: Diagrama de Sequência de Fluxo de Pagamento e Carregamento de Carteira

7 Segurança

A segurança do PayNexus é implementada em múltiplas camadas independentes, de forma a que um eventual comprometimento de uma camada não exponha o sistema inteiro.

7.1 Autenticação OAuth2/OIDC com Keycloak

O Keycloak 26.1 é o provedor de identidade central da plataforma, configurado com **dois realms independentes** [2, 3]:

- **Realm microservices** (`keycloak.pt`): utilizado pela aplicação principal e pelo IAM Service para autenticar os utilizadores da carteira;
- **Realm de pagamentos** (`payment-keycloak.pt`): utilizado exclusivamente pelo Payment Service, garantindo isolamento total entre os dois contextos de autenticação.

Esta separação implica que uma violação de credenciais no *realm* de pagamentos não compromete os utilizadores da carteira, e vice-versa. O custo é a duplicação de configuração, mitigada pela exportação de *realms* como ficheiros JSON versionados (`realm-export.json`).

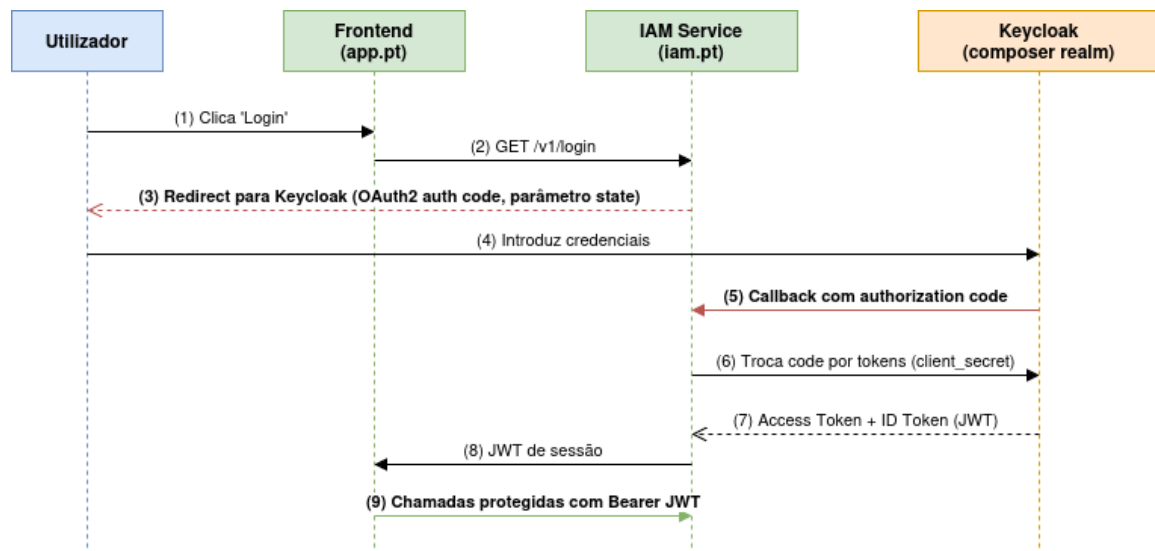


Figura 3: Fluxo de Login OIDC (OAuth2 Authorization Code Flow)

7.2 Gestão de Segredos com HashiCorp Vault

Nenhum segredo é codificado na aplicação ou incluído nas imagens Docker. O HashiCorp Vault 1.17 [4, 5] é o repositório centralizado de segredos, com o padrão **AppRole** + **agent sidecar**:

1. Cada serviço tem um `vault-agent` dedicado com `role_id` e `secret_id` próprios (AppRole);
2. O agente autentica-se no Vault ao arrancar e renderiza o template `vault/templates/<serviço>.env.tpl`;
3. O ficheiro `.env` resultante é montado em volume partilhado com o serviço;
4. O processo do serviço lê as variáveis de ambiente sem nunca aceder diretamente ao Vault.

Tabela 8: Segredos geridos por serviço no Vault

Serviço	Segredos injetados
payment_service	STRIPE_SECRET_KEY, STRIPE_WEBHOOK_SECRET TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN DATABASE_URL, INTERNAL_API_KEY, credenciais Keycloak
transactions_service	MASTER_KEY_FOR_WALLET (chave AES para carteiras) NOTIFICATIONS_API_KEY, APP_INTERNAL_API_KEY BLOCKCHAIN_NODE_URL
notifications_service	MASTER_ADMIN_SECRET, JWT_SECRET, DATABASE_URL
iam_service	CLIENT_SECRET (Keycloak)
Bases de dados	Password PostgreSQL por cada instância (payment_postgres, transaction_postgres, notifications_postgres)

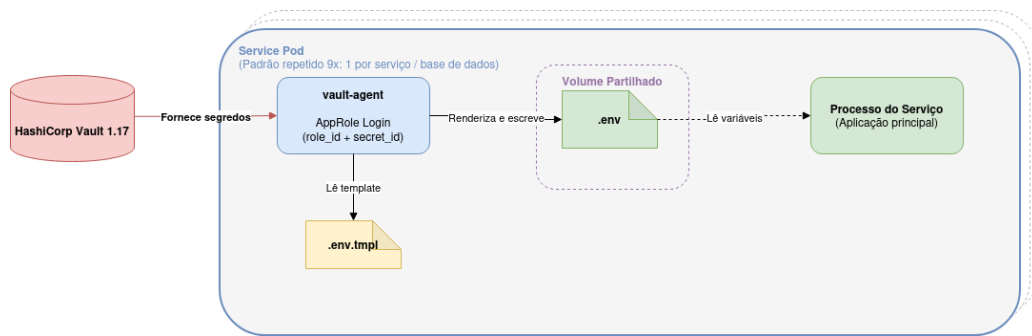


Figura 4: Padrão Vault AppRole com Agent Sidecar

7.3 Outros Mecanismos de Segurança

Tabela 9: Mecanismos de segurança adicionais

Mecanismo	Aplicação
RBAC (<i>role operator</i>)	Endpoints de administração (estatísticas, criação de clientes) apenas acessíveis com papel <i>operator</i> no JWT
Verificação HMAC <i>webhook</i>	Cabeçalho <code>Stripe-Signature</code> verificado com a chave <code>STRIPE_WEBHOOK_SECRET</code> antes de processar qualquer evento
Encriptação AES de carteiras	Chaves privadas Ethereum encriptadas com <code>MASTER_KEY_FOR_WALLET</code> antes de persistir na base de dados
OTP com hash SHA-256	Código OTP armazenado apenas como hash; expiração de 5 minutos; flag <code>used</code> impede reutilização
Isolamento de rede	Redes de bases de dados com <code>internal: true</code> inacessíveis pelo Traefik ou pela rede pública
VAPID por <i>tenant</i>	Isolamento criptográfico entre clientes nas notificações Web Push

8 Containerização e Orquestração

8.1 Docker Compose (Desenvolvimento)

Em desenvolvimento, a plataforma é orquestrada com Docker Compose num ficheiro raiz que define **17 contentores**, 4 redes e 6 volumes nomeados [6]. Todos os serviços de aplicação usam *multi-stage builds*:

Tabela 10: Imagens Docker por serviço e estratégias de *build*

Serviço / Componente	Imagem de Build	Imagem de Runtime
Transactions (Java)	maven:3.9-eclipse-temurin-21	eclipse-temurin: 21-jre
Payment Service (Python)	—	python:3.11-slim
Notifications (Go)	golang:alpine	alpine
Transactions Frontend (React)	node:20-slim	nginx:alpine
Payment Frontend (React)	—	node:20-slim

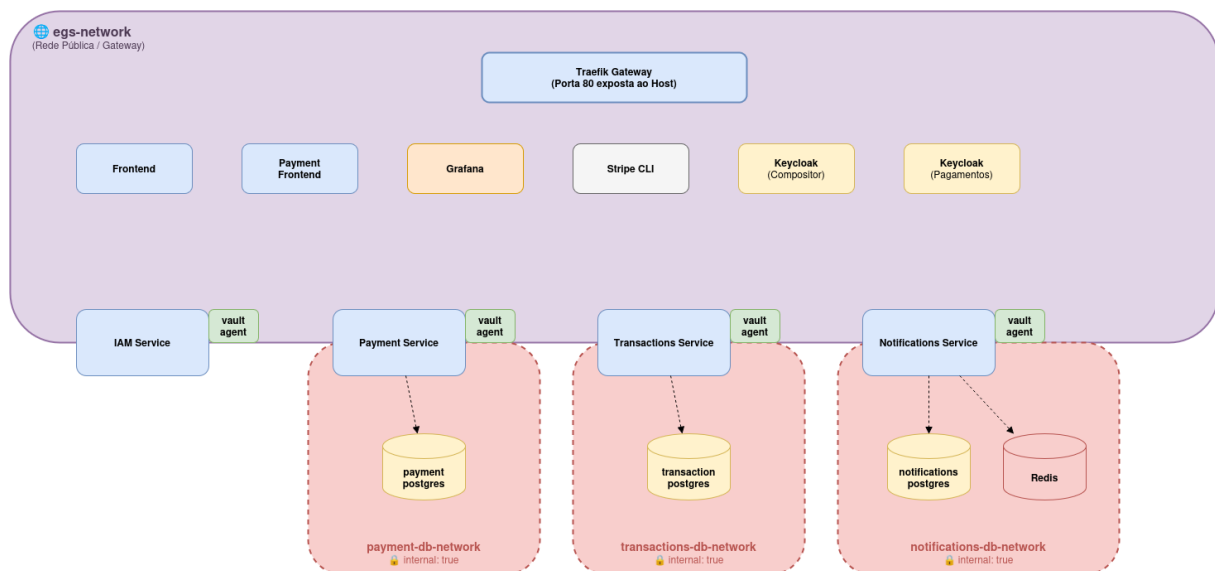


Figura 5: Topologia de Redes Docker Compose

8.2 Kubernetes (Produção)

Em produção, a plataforma corre no cluster Kubernetes do DETI/UA, no namespace `tenant-grupo3-egs-deti-ua-pt` [7]. Os manifestos disponíveis cobrem:

Tabela 11: Manifestos Kubernetes disponíveis

Manifesto	Recursos definidos
k8s/transactions-frontend.yaml	Deployment (2 réplicas) ConfigMap (VITE_TRANSACTIONS_BASE_URL) Service ClusterIP Traefik Ingress (transactions-frontend.grupo3-egs-deti-ua.pt)
k8s/notifications.yaml	Notifications API Deployment (2 réplicas) PostgreSQL Deployment + PVC (1 Gi) Redis Deployment 4 Services ClusterIP + K8s Secret Traefik Ingress (notifications.grupo3-egs-deti-ua.pt)
k8s/grafana.yaml	Grafana Deployment 3 ConfigMaps: datasources, providers, 4 dashboards JSON K8s Secret, Service ClusterIP Traefik Ingress (grafana.grupo3-egs-deti-ua.pt)

A Tabela 12 compara as duas estratégias de orquestração.

Tabela 12: Docker Compose vs. Kubernetes

Aspeto	Docker Compose	Kubernetes
Uso	Desenvolvimento local e demonstrações	Produção (cluster DETI)
Escalamento	Manual	Declarativo via replicas
Roteamento	Contentor Traefik	Traefik Ingress Controller
Segredos	Vault agents → .env	K8s Secrets
Armazenamento	Volumes nomeados	PersistentVolumeClaims
Âmbito	≈17 contentores, um <i>host</i>	Namespace, distribuído por nós
Limites de recursos	Não definidos	CPU/memória por contentor

9 Observabilidade

A observabilidade da plataforma é garantida pelo **Grafana 10** [8], disponível em `grafana.pt` (desenvolvimento) e `grafana.grupo3-egs-deti-ua.pt` (produção).

Datasources: três instâncias PostgreSQL independentes são configuradas como fontes de dados — uma por serviço com dados (transações, pagamentos, notificações). A ausência de uma base de dados partilhada obriga a que cada dashboard consulte a sua própria instância, refletindo a separação de domínios da arquitetura.

Dashboards disponíveis:

- **main_dashboard**: visão global do sistema — transações realizadas, notificações enviadas, pagamentos concluídos;
- **payments_dashboard**: volume de pagamentos, receita, taxa de sucesso, estatísticas OTP, padrões horários (18 painéis);
- **transactions_dashboard**: métricas *blockchain* — transações on-chain, saldos, taxas de gas;
- **notifications_dashboard**: entrega de notificações, falhas, canais ativos.

Provisionamento: todos os datasources e dashboards são declarados como configuração (ficheiros YAML e JSON), sem necessidade de configuração manual na UI. Em Kubernetes, este provisionamento é encapsulado em ConfigMaps, garantindo que o Grafana arranque totalmente configurado.

10 Desafios e Conclusões

10.1 Desafios Encontrados

Tabela 13: Desafios e aprendizagens do projeto

Desafio	Aprendizagem
Operações políglota	4 linguagens = 4 cadeias de build/runtime para containerizar e manter consistentes
Segredos à escala	Vault + <i>agent sidecars</i> eliminou segredos do código, mas acrescentou componentes móveis ao sistema
Dois <i>realms</i> Keycloak	Isolamento forte entre utilizadores da carteira e do pagamento, ao custo de duplicação de configuração
Pagamentos por <i>webhook</i>	Confiar nos <i>webhooks</i> da Stripe (não no cliente) obriga a abraçar a consistência eventual
<i>Testnet</i> Ethereum	Transações reais na Sepolia requerem <i>gas</i> , encriptação de chaves e um <i>mock provider</i> para desenvolvimento
<i>Gateway</i> único	O Traefik simplificou a superfície de ataque — uma porta exposta, tudo o resto interno

10.2 Conclusões

O projeto **PayNexus** demonstra com sucesso que é viável implementar um sistema *fin-tech* com carregamento direto de dinheiro para a *blockchain* usando uma arquitetura de microsserviços. A separação em *bounded contexts* independentes permitiu usar a melhor ferramenta para cada domínio: Python para as APIs web, Java para a integração *blockchain* com tipagem forte, e Go para o motor de notificações de alto débito.

A segurança por camadas, assente na federação de identidades, isolamento estrito de rede, validação de *webhooks* e centralização de segredos, garante que nenhum vetor de ataque isolado compromete a integridade total do ecossistema, blindando os dados sensíveis dos utilizadores.

Em suma, a concretização deste projeto validou na prática os princípios fundamentais de Engenharia e Gestão de Serviços (EGS). Mais do que a simples integração tecnológica entre o mundo financeiro tradicional e as redes descentralizadas, o **PayNexus** provou que o verdadeiro valor de uma arquitetura políglota reside na capacidade de entregar resiliência, escalabilidade e independência operacional a cada serviço, resultando num produto final robusto, seguro e totalmente pronto para ambientes de produção.

Referências

- [1] M. Thomson and P. Beverloo. Voluntary application server identification (vapid) for web push. RFC 8292, Internet Engineering Task Force (IETF), 2017. URL <https://datatracker.ietf.org/doc/html/rfc8292>.
- [2] Red Hat, Inc. Keycloak: Open source identity and access management, 2024. URL <https://www.keycloak.org/documentation>. Versão 26.1.
- [3] D. Hardt. The oauth 2.0 authorization framework, 2012. URL <https://datatracker.ietf.org/doc/html/rfc6749>. RFC 6749.
- [4] HashiCorp, Inc. Vault approle auth method, 2024. URL <https://developer.hashicorp.com/vault/docs/auth/approle>. HashiCorp Vault 1.17.
- [5] HashiCorp, Inc. Vault agent, 2024. URL <https://developer.hashicorp.com/vault/docs/agent-and-proxy/agent>. HashiCorp Vault 1.17.
- [6] Docker, Inc. Multi-stage builds, 2024. URL <https://docs.docker.com/build/building/multi-stage/>. Docker documentation.
- [7] The Linux Foundation. Kubernetes documentation, 2024. URL <https://kubernetes.io/docs/home/>. kubernetes.io.
- [8] Grafana Labs. Grafana documentation, 2024. URL <https://grafana.com/docs/grafana/latest/>. Versão 10.0.0.